

EXAM 3

$$\frac{62}{48} \quad \frac{61}{48}$$

$$\frac{1}{4} \quad \frac{1}{3}$$

February 8, 2018

I.

II min nr of bits

i) 61

$$61 = 3A_{16} = 0011 \ 1101_b$$

the min nr of bits necessary for rep 61 is 6 : 11 1101b

$$61 \in [0, 2^6] \Rightarrow 61 \in [0, 64]$$

ii) -62

$$-62 = 2^5(62)$$

$$62 = 3E_{16} = 0011 \ 1110_b \Rightarrow -62 = 1100 \ 0010_b$$

$$-62 \in [-2^6, 2^6] \Rightarrow \text{the min nr. of bits} = 7$$

(formula for signed : $[-2^{n-1}, 2^{n-1}]$)

iii) 130

$$130 = 82_{16} = 1000 \ 0010_b$$

min nr. of bits for rep. 130 is 8 : 1000 0010b

$$130 \in [0, 2^8-1]$$

iv) -129

$$-129 = 2^5(129)$$

$$129 = 81_{16} = 1000 \ 0001_b \Rightarrow -129 = 0111 \ 1111_b$$

$$= 0000 \ 0000 \ 1000 \ 0001 \Rightarrow -129 = 1111 \ 1111 \ 0111 \ 1111_b$$

$\Rightarrow$ the min nr. of bits for rep -129 is 9

$$-129 \in [-2^9, 2^9]$$

b) `xor ah, ah`; ~~ah~~ → puts in the high part 0's

`cwde`; ~~ax~~ → ~~eax~~ convert `AX` → `EAX`

`add ebx, ebx`; ~~adds to eax, ebx~~ `ebx` ← `ebx + al`

`mov al, [ebx]`; `al` ← `[ebx + al]`

($\Rightarrow$) `xlat`
(translates the byte `AL` into a corresponding byte from a translation table defined)

`[DS:EBX]`-address of translation
implicit table

III.

a) `mov ax, 1000h`

`mov bl, 1000b + 10b`

`div bl`

`ax` = `1000h` = `4096d` (signed and unsigned)

`1000b` = `8d` } $\Rightarrow$ `1000b + 10b` = `10d` = `0Ah`

`10b` = `2d` } (signed and unsigned)

`div bl` $\Rightarrow$ `AX:BL` = `4096:10` = `409` $\times$ `6`

~~$\Rightarrow$ `AX` = `409` = `0199h` (quotient)~~

~~`DX` = `6` = `0006h` (remainder)~~

~~For this case, there is not overflow. Overflow for division~~

$\Rightarrow$ `AL` = `409` = `0199h` (quotient)

`AH` = `6` = `0006h` (remainder)

but `0199h` doesn't fit a byte
so there will be an overflow
marked by a runtime error (the program
will stop after the instr ~~mov~~ `div bl`)

$$\begin{array}{r} 256 \\ 16 \\ \hline 1536 \quad 3 \\ 256 \\ \hline 4096 \end{array}$$

$$\begin{array}{r} 409 \\ 256 \\ \hline 153:16 = 9 \\ 144 \\ \hline = 9 \end{array}$$

$$130 : 16 = 8$$

$$\frac{112}{18}$$

$$\frac{128}{2}$$

b) mov ah, 0bch

mov al, 0deh

add ah, al

$$ah = 0bch = 1011 \ 1100b$$

$$al = 0deh = 1101 \ 1110b$$

add ah, al

$$\begin{array}{r} 1011 \ 1100 \\ + 1101 \ 1110 \\ \hline 1001 \ 1010 \end{array}$$

- there's a digit outside of the binary configuration of the final result (a transport digit) $\Rightarrow CF = 1$

- since, in the signed interpretation, there is an addition of two negative numbers, and the result is still a negative one $\Rightarrow$ no overflow on s. i. $\Rightarrow OF = 0$

c) mov ax, 1001h

mov bx, 1111b

imul bl

$$ax = 1001h \xrightarrow{al=1} = 4096 + 1 = 4097d$$

$$bx = 1111b = 15d \ 000fh$$

$$bl = 15 (signed \ and \ unsigned)$$

$$imul \ bl : al * bl = 1 * (-1) = -1 \Rightarrow \text{result on } AX = ffffh$$

$$OF = CF = 0 \ (b * b = b)$$

$$= 01 * 15 = 15 \Rightarrow \text{result on } AX = 000fh$$

$$OF = CF = 0 \ (b * b = b)$$

d) mov dh, 62h

mov ch, 200

sub dh, ch

$$62h = 96 + 2 = 98d \ (\text{signed and unsigned}) \ 0110 \ 0010$$

$$200 = c8h = 1100 \ 1000b = 200 (unsigned), -56 (signed)$$

$$\begin{array}{r} 16 \\ 128 \\ \hline 16 \\ 40 \\ \hline 32 \end{array}$$

$$200 : 16 = 12$$

Sub dh, eh

$$\begin{array}{r} \text{CF}=1 \\ 0110 \ 0010 - \\ 1100 \ 1000 \\ \hline 0000 \ 1010 \end{array}$$

→ for the subtraction, there was needed a borrow
so CF = 1
→ pos - neg = pos (no overflow on signed int) →
⇒ OF = 0

In assembly language, an overflow means that a result of a performed operation didn't fit in the admissible size representation or that the result doesn't belong to the specified interval or that the resulted operation is incorrect from the ~~point of view~~ mathematical point of view.

The overflow for unsigned numbers is marked by the CF when there appears a carry outside the resulted binary config.

ex. - -

subtraction (borrowed)

$$\begin{array}{l} \text{odd} \\ \text{OF} \rightarrow \left\{ \begin{array}{l} \text{neg} + \text{neg} = \text{pos} \\ \text{pos} + \text{pos} = \text{neg} \end{array} \right. \\ \text{sub} \end{array}$$

$$\text{CF} \rightarrow \left\{ \begin{array}{l} \text{neg} - \text{pos} = \text{pos} \\ \text{pos} - \text{neg} = \text{neg} \end{array} \right.$$

OF = CF for mult (but there's no overflow result = 2 * size of op)

for overflow div ⇒ runtime error

February 9, 2018

I Overflow concept

Overflow rep a situation (condition) in which the result didn't fit in the reserved size space OR the res. doesn't belong to the arith. rep. interval or the operation is a math. nonsense in that particular interpretation (signed/unsigned)

- Addition ($CF = a+b$) internal, digit outside, $OF=1$ (neg+neg=pos, pos+pos=neg)
 - Subtraction (~~CF~~ $CF=1$ (needs a borrow digit), $OF=1$ (pos-neg=neg, neg-pos=pos))
 - Multiplication (No overflow, but $CF=OF=0: b*b=b$, $CF=OF=1: b*b=wr$)
 - Division (res don't fit on quotient $\Rightarrow$ "Run-time error" by OS "providing one of: "Div overflow", "~~no division~~" "Div by zero", "zero divide")
- $\rightarrow$ The prog. • should be careful about ~~the~~ allocating the right size for every explicit data, by using a suitable data type every time
- checking the flags after every execution (CF, OF)
- $\rightarrow$ The solving of these ~~not~~ situations mean changing the data type, so that is respecting the saved size for every

II. a1 db '256, -256' | 32 35 36 '-' 32 35 36

$\rightarrow$ in mem there are placed the ascii codes for each char in a byte

$\Rightarrow$ memory : 32 35 36 '-' 32 35 36

'2' = 50 = 32h

'5' = 53 = 35h

'6' = 54 = 36h

a2 dw 256, 256h | 00 01 56 02

256 = ~~0010~~h 0100h $\Rightarrow$ memory: ~~0010~~ 00 01

256h = 0256h $\Rightarrow$ memory: 56 02

a3 dw \$-a2 | 04 00

$$\$ - a2 = 10 - 6 = 4 (\text{no. of bytes between}) = 0004h$$

⇒ memory: 04 00

a4 equ -254/4

→ nothing will be shown in memory (equ-directive for initializing with constants without allocating memory space)

a5 db 128 >> 1, -128 << 1 | 40 00

(doesn't fit a byte)

$$128 = 1000\ 0000 \Rightarrow 128 \gg 1 = 0100\ 0000 = 40h \Rightarrow \text{memory } 40$$

$$-128 = 2^5(128) = \text{too big for a word } 1000\ 0000 \Rightarrow -128 \ll 1 = 0000\ 0000 \Rightarrow \text{mem: } 00$$

a6 dw a2-a5, ~ (a2-a5) | FF FF 05 00

$$a2 - a5 = 8 - 14 = -6$$

$$-6 = 2^5(6)$$

$$4 = 0000\ 0000\ 0000\ 0110 \Rightarrow -4 = 1111\ 1111\ 1111\ 1001 = FFFAh \Rightarrow$$
$$\Rightarrow \text{memory: } FF FF$$

$$\sim(a2-a5) = \sim(-6) = \sim(1111\ 1111\ 1111\ 1010) = 0000\ 0000\ 0000\ 0110 = 0005h \Rightarrow$$
$$\Rightarrow \text{memory: } 05\ 00$$

a7 dd [a2], !a2 - syntax error (not computable at assembly time
a var's addr not comp. at ass time)

!a2 = ! (addr of a2 ≠ 0) = 0, but not shown in mem. since syntax error
! → address

a8 dd 256h ^ 256, 256256h | 56 03 00 00 56 62 25 00

$$256h = 0000\ 0256h = 0000\ 0000\ 0000\ 0000\ 0000\ 0010\ 0101\ 0110$$

$$256 = 0000\ 0100h = 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0000\ 0000 \Rightarrow 256h \wedge 256$$

$$= 0000\ 0000\ 0000\ 0000\ 0000\ 0011\ 0101\ 0110 = 0000\ 0356h \Rightarrow$$

memory: 56 03 00 00

256 256 h = 00 256 256 h $\Rightarrow$ memory: 56 62 25 00

a9 dd ($\phi - a8$) + ($a10 - \phi$) | 00 00 00 00

$\phi - a8 = 8(\text{m. of bytes} \dots)$
 $a10 - \phi = -\cancel{12} + 4(a10 - \phi)$
 $(\phi - a8) + (a10 - \phi) \rightarrow$ will be a dword $\Rightarrow$ 4 bytes
 $\Rightarrow (\phi - a8) + (a10 - \phi) = 12 = 0004h \Rightarrow$ memory: 04 00
00 00 00 00 h $\Rightarrow$ memory: 04 00 00 00

a10 dw -255, 256 | 01 FF 00 01

-255 = $2^8(255)$

$\overbrace{255}^{(2^8-1)} = 0000 \ 0000 \ 1111 \ 1111 \Rightarrow -255 = 1111 \ 1111 \ 0000 \ 0001 = FF01h \Rightarrow$ memory: 01 FF

$256 = 2^8 = 0000 \ 0001 \ 0000 \ 0000 = 0100h \Rightarrow$ memory: 00 01

a11 resb 6 | 00 00 00 00 00 00

$\rightarrow$ reserves 6 bytes $\Rightarrow$ memory

a12 times 4 dw 256 | 00 01 00 01 00 01 00 01

$256 = 0100h \Rightarrow$ memory: ~~00 00~~ 00 01

a13 dw times 4, -128; syntax error

~~00 00~~

times 2 resw 2 | 00 00 00 00 00 00 00 00

$\rightarrow$ res 2×2 words

times 2 dd 12345678h | 78 56 34 12 78 56 34 12

12345678h $\Rightarrow$ memory: 78 56 34 12

MEMORY LAYOUT

32 35 36 ' ' ' 32 35 36 00 01 56 02 04 00 40 00 FF
 a1 a2 a3 a5

~~FF~~ 06 00 56 03 00 00 56 62 25 00 0A 00 00 00 0A
 a6 a8 a9 a10

~~FF~~ 00 01 00 00 00 00 00 00 00 01 00 01 00 01 00
 a11

01 00 00 00 00 00 00 00 00 00 48 56 34 12 48 56 34 12
 a12

EBX = 0034C000

II a)

- 1) lea ebx, [ebx+6] 0034C006
- 2) lea ebx, [bx+6] → signed extension 0000C006
- 3) lea bx, [bx+6] 0034C006
- 4) lea bx, [ebx+6] 0034C006
- 5) mov ebx, ebx+6 - (SE)
- 6) mov ebx, [ebx+6]; tries to load in ebx the content from the address ebx+6 (memory violation error)
- 7) movzx ebx, [ebx+6] - (SE?)
- 8) movzx ebx, [bx+6] → unsigned ex - (SE) ([bx+6] - offset on 32 bits)
- 9) add bx, 6; movzx ebx, bx 0000C006
- 10) ^{size}mov [ebx], dword [bx+6] - (SE)
- 11) add ebx, 6 0034C006
- 12) add bx, 6 0034C006
- 13) ^{size}push [ebx+6]; pop ebx (SE)
- 14) xchg ebx, [ebx+6]; tries to put in ebx the content from address ebx+6 (memory violation error)

I modifies entire ebx (refers to entire ebx)

1), 3), 4), 11), 12)

II modifies only bx and the rest = 0

2), 9)

III memory violation error

6), 14)

IV syntax error

5), 4), 8), 10), 13)

b) `add`

`push ax`

`mov ax, [esp+2]; mov ax, ax`

`stosw`; `mov edi, ax`; take ax from the stack and stores it in edi

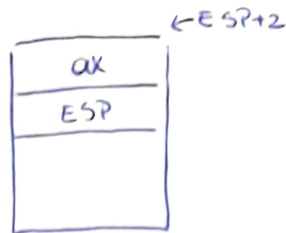
`lea edi, [edi-2]; mov edi, ax`

`add esp, 4`

`stosw`; loads in EDI → AX

~~EDI ← EDI + 2~~

`EDI ← EDI + 2`



EXAM 2

January 28, 2019

I - using 2's complement (def.)

- 4 reguli (din $\overline{100...0}$ - signed, $\overline{0...0}$ - signed, $\overline{100...0} = abc \Rightarrow ? -abc$
 $\overline{00...0} = abc \Rightarrow ? abc$)

$2^m - V$ - 2's complement

- addition, subtraction the same for signed/unsigned
- multiplication (mul), division (div)
- 0 - considered a positive nr (if cons. its signed ~~to~~ interp. the sign bit is 0 which translates it as a positive number)

II.

a1 db '256' | ^{32 35 36} ~~50 53 54~~ '1' = 49
(offset = 004001000)

'256' - string - stored as '2' '5' '6' $\Rightarrow$ memory: 50 53 54
^{50 = 32h 53 = 35h 54 = 36h}

a2 dw 256, 256h | ^{00 40} 56 02 (offset = 004001003)

256 = 0100h $\Rightarrow$ memory: 00 40

256h = 0256h $\Rightarrow$ memory: 56 02 ^{1/2}

a3 dw ~~+~~ a2; syntax error (pointer addition not possible)
(possib. mem. violation error)

a4 equ -256/4 | nothing shown in memory, equ is a directive used for
a4 = -64; initialising a variable with a constant without
; allocating memory space

a5 db 256 >> 1, 256 << 1 | 80 00 ✓

256 = $2^8 = 1$ | 0000 0000

256 >> 1 = 1000 0000 = 80h $\Rightarrow$ memory: 80

256 << 1 = 0000 0000 = 00h $\Rightarrow$ memory: 00

$$a6 \text{ dw } a5-a2, !(a5-a2) \mid 04 \text{ } 00 \text{ } 00 \text{ } 00$$

$$a5-a2 = 8-6 = 2 \text{ (nr. of bytes between } a5 \text{ and } a2) \Rightarrow \text{memory: } 04 \text{ } 00$$

$$!(a5-a2) = !(2) = 0 \Rightarrow \text{memory } 00 \text{ } 00$$

$a7 \text{ dw } [a2], \text{ var2}$; syntax error, not determinable at assembly time

$$a8 \text{ dd } 256h \wedge 256, 256256h \mid 56 \text{ } a3 \text{ } 00 \text{ } 00 \text{ } 56 \text{ } 62 \text{ } 25 \text{ } 00$$

$$256h = 0000 \text{ } 0256h = 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0010 \text{ } 0101 \text{ } 0110$$

$$256 = 0000 \text{ } 0010h = 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000$$

$$\Rightarrow 256h \wedge 256 = 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0000 \text{ } 0010 \text{ } 0101 \text{ } 0110 = 0000 \text{ } a3 \text{ } 56h$$

$$\Rightarrow \text{memory: } 56 \text{ } a3 \text{ } 00 \text{ } 00$$

$$256256h = 0025 \text{ } 62 \text{ } 56h \Rightarrow \text{memory} = 56 \text{ } 62 \text{ } 25 \text{ } 00$$

$$a9 \text{ dd } \$-a9 \mid 0000 \text{ } 00 \text{ } 00$$

$$\$-a9 = 0 \text{ (current offset)}$$

$$a10 \text{ db } 256, -255 \mid 00 \text{ } ff$$

$$256 = 2^8 = 1 \mid 0000 \text{ } 0000 = 00h \text{ (one byte)}$$

$$-255 = 2^8(255) \quad -255 = 2^8(255) = 1(256-255)$$

$$255 = 1111 \text{ } 1111 = ffh$$

$$a11 \text{ dw } 256h-256 \mid 56 \text{ } a4$$

$$256h = 2 \cdot 256 + 5 \cdot 16 + 6 = 512 + 80 + 6 = 598d$$

$$598 - 256 = 342d = 2 \cdot 256 + 5 \cdot 16 + 6 = 0156h \Rightarrow \text{memory: } 56 \text{ } a4$$

$$\Delta 12 \text{ dw } 256 - 256h \mid AA \neq E$$

$$256h = 598d$$

$$256 - 598 = -342$$

$$-342 = 2's(342)$$

$$342 = 0156h = 0000\ 0001\ 0101\ 0110 \Rightarrow 2's(342) = 1111\ 1110\ 1010\ 1010$$

$$= FEAA \Rightarrow \text{memory} : AA \neq E$$

$$\Delta 13 \text{ dw } -256 \mid 00 \neq F$$

$$-256 = 2's(256)$$

$$256 = 0000\ 0001\ 0000\ 0000 \Rightarrow 2's(256) = 1111\ 1111\ 0000\ 0000 = FF00h$$

$$\Rightarrow \text{memory} : 00 \neq F$$

$$\Delta 14 \text{ dw } -256h \mid AA \neq D$$

$$-256h = - (0000 \underbrace{0010}_{512} \underbrace{0101}_{80} \underbrace{0110}_6) = -598$$

$$-598 = 2's(598)$$

$$598 = 0000\ 0010\ 0101\ 0110 \Rightarrow 2's(598) = \cancel{1111} \cancel{1110} 1111\ 1101\ 1010\ 1010$$

$$= FDAA \Rightarrow \text{memory} : AA \neq D$$

$$\begin{array}{l} 342 : 16 = 21 \text{ r } 6 \\ \underline{320} \\ 22 \\ \underline{16} \\ 6 \end{array} \quad \begin{array}{l} 598 : 16 = 37 \text{ r } 6 \\ \underline{576} \\ 22 \\ \underline{16} \\ 6 \end{array} \quad \begin{array}{l} 342 : 16 = 21 \text{ r } 6 \\ \underline{320} \\ 22 \\ \underline{16} \\ 6 \end{array} \quad \begin{array}{l} 598 : 16 = 37 \text{ r } 6 \\ \underline{576} \\ 22 \\ \underline{16} \\ 6 \end{array}$$

015 ab 2,5,6,25,6,2,56 | 02 05 06 19 06 02 38

$$25 = 19h$$

$$56 = 38h$$

(a) CF, AH, SF, OF, ZF

a1) mov^{ah}, 129

mov bh, 9Fh

add ah, bh

$$ah = 129 = 81h = 1000\ 0001$$

$$bh = 9F = 144 + 15 = 169 = 1001\ 1111$$

$$129 + 169 = 298$$

$$\begin{array}{r} 11\ 111 \\ 1000\ 0001 + \\ 1001\ 1111 \\ \hline 1001\ 0000 \end{array}$$

the result doesn't fit a byte (there's a transp. digit) $\Rightarrow$

$$\Rightarrow CF = 1$$

$$SF = 0 \text{ (MSB = 0)}$$

OF = 1 (~~the~~ 2 ~~positive~~ ~~neg~~ numbers - starting with 1' addition provide a positive result, which is math. impossible)

$$ZF = 0 \text{ (the result } \neq 0)$$

$$AH = 20h$$

a2) mov ax, 128

shr al, 4

imul ah

$$ax = 128 = 2^7 = \overbrace{0000\ 0000}^{ah} \overbrace{1000\ 0000}^{al}$$

shr al, 7 $\Rightarrow$ 1111 1111 (complete with the sign bit)

$$= FFh = 255 = -1$$

$$\text{imul ah} \Rightarrow \underbrace{al}_{-1} * \underbrace{ah}_0 \Rightarrow AX = 0 \quad \left\{ \begin{array}{l} CF = OF = 0 \text{ (no overflow } b * b = b) \\ SF = 0 \text{ (0)} \\ ZF = 1 \text{ (the result is 0)} \end{array} \right.$$

zero flag is not set
after division/multiplication)

a3) ~~mov~~ mov ax, 256

mov bx, -1

add ah, bh

$$ax = 256 = 2^8 = \overbrace{0000\ 0001}^{ah}\ 0000\ 0000$$

$$bx = -1 = \overbrace{1111\ 1111}^{bh}\ 1111\ 1111$$

$$-1 = 2^15(1) = 1111\ 1111\ 1111\ 1111$$

$$\begin{array}{r} 1111\ 1111 \\ 0000\ 0001 \\ 1111\ 1111 \\ \hline 10000\ 0000 \end{array} \Rightarrow$$

$$\left\{ \begin{array}{l} CF = 1 \text{ (there's a transp digit outside the rep. ~~unit~~ ^{swing})} \\ OF = 0 \text{ (no overflow on signed, } 0 \in [-128, 127]) \\ SF = 0 \\ ZF = 1 \end{array} \right.$$

a4) mov ah, 128/2

mov bh, 90h >> 3

sub ah, bh

$$\left. \begin{array}{l} 128 = 1000\ 0000 \\ 2 = 0000\ 0010 \end{array} \right\} \textcircled{1} \Rightarrow ah = 1000\ 0010$$

$$90h = 1001\ 0000 \xrightarrow{\gg 3} 0001\ 0010 \Rightarrow bh = 0001\ 0010$$

sub ah, bh

$$\begin{array}{r} 1000\ 0010 \\ - 0001\ 0010 \\ \hline 0001\ 0000 \end{array}$$

$$\left\{ \begin{array}{l} CF = 0 \text{ (there it is needed to borrow a digit so CF will be set to 1)} \\ OF = 1 \text{ (~~no overflow on signed~~ } 16 \in [-128, 127] \text{)} \\ SF = 0 \text{ (neg-pos = pos)} \\ ZF = 0 \end{array} \right.$$

b1) `movzx, operand1, operand2` `movzx eax, al`; puts in `eax` the unsigned extension of `al`
• for this instruction both operands should be of different sizes.
in the first one it is loaded the unsigned-extended ^(register) one, so the first one must be content of the second one (reg/memory ~~at~~ location)

b2) `cwv`; the implicit operands are `al, ax`
• for this instruction both implicit operands are of different sizes, the first one (`al`) ^{-which is a byte} is converted to a word (`ax`).

b3) `xlat`; the implicit operands are `al, table`.

b3) IT DOESN'T EXIST! THERE CANNOT BE 2 OPERANDS FROM MEMORY because there's only 1 [Mode R/M] which specifies the type of operand (Register or memory ~~or~~ location) in the instruction ~~assembly~~ address computation.

b4) `movsb` ~~loads~~
• loads into `EDI` the content of `ESI`

b5) ~~`cmp op1, op2`~~ `movsb`
3

February 18, 2019

A flag is an indicator of 1 bit representation. The EFLAGS register ~~is a 32~~ shows a synthetic overview of (the effects made by the instructions performed) the execution of the instructions. For the 80x86 microprocessor, the EFLAGS register (status register) is a 32 bit register, but only 9 of them are used. The flags are: CF, PF, AF, ZF, SF, TF, IF, DF, OF. Their classification may be structured on 2 categories:

- flags that ~~are not~~ are implicitly set by the processor (that show a previous effect): CF, PF, AF, ZF, SF, OF

- flags that can be explicitly set by the programmer (for a future effect):

CF, IF, DF, TF. But there is no direct access to TF, for the fact that an accidental set of this one can determine the stop of the execution, ~~so~~ this is why TF is a debugger flag (and why there are debuggers even step by step)

- CF (Carry Flag) - is the transport flag. It is set to 1 if in the Last Performed Operation, there was a transport digit outside the representation domain, and 0 otherwise

- this flag marks the unsigned overflow

the carry flag will be set to 1 since the result doesn't fit ~~a~~ 1 byte.

$$\begin{array}{r} 11110110 + \\ 11111111 \\ \hline 111101101 \end{array}$$

- PF (Parity Flag) - it is set to 1 if the number of 1's in the binary representation of the Last Performed Operation is even, and set to 0 otherwise

ex: if the LPO = 11001100 $\Rightarrow$ PF = 1

- ~~Also~~ AF (Auxiliary Flag) - it is ~~is~~ set to 1 if in the last performed operation there was a transport digit from bit 3 to bit 4

- if in an addition there was a carry from the low nibble to the high nibble or if in a subtraction there was a

borrowed from the high nibble to the low nibble.

ex:
$$\begin{array}{r} 1000 \overset{1}{1111} + \\ 0000 \quad 1111 \\ \hline 1001 \quad 1110 \end{array}$$

- ZF (Zero Flag) - it is set to 1 if the result of the LPO = 0, and set to 0 otherwise
- SF (Sign Flag) - it stores the most sign bit of the LPO (sign bit)
(1 - negative, 0 - positive)
- Trap Flag (TF) - is a debug flag
 - if set to 1 $\Rightarrow$ stops after every execution
 - if set to 0 $\Rightarrow$ let everything flow
- IF (Interrupt Flag) - if set to 1 interrupts are allowed
 - critical section = section of code that cannot be interrupted
 - the time between the switch of IF = true critical section
- DF (Direction Flag) - ~~if set to 1~~ for operating with string instructions
 - if set to 1 $\Rightarrow$ the string is parsed in descending order (end $\rightarrow$ start)
 - if set to 0 $\Rightarrow$ the string is parsed in ascending order (start $\rightarrow$ end)
- OF (Overflow Flag) - it marks the signed overflow
 - set to 1 if the ~~rep~~ binary rep. of the last pref. op didn't fit the rep. domain, 0 otherwise

considered in signed interpretation

The instr. that are most involved with the flag values are `imul`, `idiv` because these instructions

The conf. of EFLAGS can be modified by pushing the content onto the stack and restore the EFLAGS register (by popping the top of the stack)

POPF

The flags reacting to an overflow situation are CF, OF

CF - set to 1 if there's a transp. digit outside the rep. domain

OF - set to 1 if the res of LTO doesn't fit the internal rep. cons. in signed interpretation or an operation is not providing a logical mathematical result

- when positive + positive = negative ($0... + 0... = 1... -$) OF=1
- when neg + neg = positive ($1... + 1... = 0$) OF=1
- when ~~positive~~ negative = neg - pos = pos
- when pos - neg = neg

There is more than one flag dealing with an overflow situation, because there are two interpretations of every representation (unsigned and signed) so there are 2 possible results which can be ~~separated~~ interpreted based on how the following instructions may be referring to. There are 2 diff int of the admissible internal repr., even though there are a certain n. of values ~~in an~~ for a data type (byte, word, ...) but since on unsigned the int. $[0, 2^n - 1]$, n-m. of bits and in signed $[-2^{n-1}, 2^{n-1} - 1]$ the overflow situation is different for every interpretation, so we need 2 flags.

• instructions operating with flags (CF, OF, AF, DF)

mov AH, 200; AH = C8

mov BH, 200; AH = C8

add AH, BH; in AH it will be tried to be add $200+200=400$, but this
; doesn't fit in a byte, so instead AH = 144, which is an
; incorrect result generated because of the overflow, so in this
; case CF = 1

mov AH, ~~50~~ 1010 0000b

mov BH, ~~50~~ 1010 0000b

add AH, BH; AH = 0100 0000; in this case, in signed interpretation, it can
; be seen that 2 negatives provide a positive
; result, which is mathem. incorrect, so OF = 1
; also $1010\ 0000b = -96d$, $-96 - 96 = -192 \notin [-128, 127]$

mov AH, 1010 0000b

mov BH, 1010 0000b

add AH, BH; AH = 1101 0000; AF = 1

std; sets the DF = 1

→ The instructions for direct accessing the flags are:

stc → CF = 1

cnc → CF = 0

~~cnc~~ cmc → CF = $\neg$ CF

std → DF = 1

cld → DF = 0

sti → IF = 1

cld → IF = 0

a) x dw -256, 256h | 00FF 56 02

$$-256 \rightarrow 2's(256)$$

$$256 = 0000\ 0001\ 0000\ 0000$$

$$256 = 0000\ 0001\ 0000\ 0000$$

$$2's = 1111\ 1111\ 0000\ 0000 = FF00h \Rightarrow \text{memory } 00\ FF$$

$$256h = 0000\ 0010\ 0101\ 0110 = 0256h \Rightarrow \text{memory } 56\ 02$$

b) y dw 256|-256, 256h & 256 | 00 FF 00 00

$$256 = 0000\ 0001\ 0000\ 0000$$

$$-256 = 1111\ 1111\ 0000\ 0000$$

$$256|-256 = 1111\ 1111\ 0000\ 0000 = FF00 \Rightarrow \text{memory } 00\ FF$$

$$256h \& 256$$

$$256h = 0000\ 0010\ 0101\ 0110$$

$$256h = 0000\ 0010\ 0101\ 0110$$

$$256 = 0000\ 0001\ 0000\ 0000$$

$$256h \& 256 = 0000\ 0000\ 0000\ 0000 = 0000h \Rightarrow \text{memory } 00\ 00$$

c) z db \$-z, y-x | 00 04

$$\$-z = 0 (\$ - \text{current offset}, - \text{offset of } z) \Rightarrow \text{memory } 00$$

$$y-x = 4 (4 \text{ bytes difference}) \Rightarrow \text{memory } = 04$$

d) db 'y'-'x', 'y-x' | 01 44 ascii('-') 46

$$'y'-'x' = 1 (\text{the diff between the ascii codes}) \Rightarrow \text{memory } 01$$

$$'y-x' = \text{ascii}(y) - \text{ascii}(x) \Rightarrow \text{memory } 44 \text{ ascii}('-') \ 46$$

$$\text{ascii}(y) = 121 = 44h$$

$$\text{ascii}(x) = 120 = 46h$$

k) a db 512 >> 2, -512 << 2 / 80 00

$512 = 2^9$ = ~~no~~ doesn't fit a byte $\Rightarrow$ rep. on a word = 0000 0010 0000 0000
 $(2^9 : 2^8) = 2^1$
 $\Rightarrow 0000 0000 1000 0000 = 0080 \Rightarrow$ byte 80 $\Rightarrow$ memory 80

$-512 = -2^9 \Rightarrow$ rep. on a word = 1111 1110 0000 0000

$-512 << 2 = 1111 1000 0000 0000 = \text{F8 00} \Rightarrow$ memory 00

f) b dw 2-a, !(2-a) | FA FF 00 00

$2-a = 10-16 = -6$ (-nr. of bytes between 2 and a)

$-6 = 2^5(6) = \text{FFFA h} \Rightarrow$ memory: FA FF

$6 = 0000 0000 0000 0110$

$-6 = 1111 1111 1111 1010 \Rightarrow !(6) = 0000 0000 0000 0101 = 0006 \text{ h}$
 ~~$\Rightarrow$ memory 06 00~~ $!(-6) = 0 = 0000 \text{ h} \Rightarrow$ memory 00 00

g) c dd (\$-b)+(d-\$), \$-2*y+3 syntax error ϕ sta pc loc

$\phi - b = 4$

$d - \phi = 4 \} \rightarrow$ ~~memory~~ 0000 0008 $\Rightarrow$ 00 00 08 00

~~$\phi - 2*y + 3$~~

~~$2*y =$ an unknown address~~ $\} \rightarrow$ ~~memory violation error (probably)~~

$2*y$ is not allowed $\Rightarrow$ syntax error

h) d db -128, 128 ^ (~128) / 80 FF

$-128 = 2^5(128)$

$128 = 2^7 = 1000 0000 = 80 \text{ h}$

$-128 = 1000 0000 = 80 \text{ h} \Rightarrow$ memory: 80

$128 \wedge (\sim 128)$

$\sim 128 = \sim(1000 0000) = 0111 1111$
 $128 = 1000 0000 \} \Rightarrow 1111 1111 = \text{FF h} \Rightarrow$ memory FF

a) e times 2 resw 6

⇒ memory $\underbrace{00\ 00}_1 \underbrace{00\ 00}_2 \underbrace{00\ 00}_3 \underbrace{00\ 00}_4 \underbrace{00\ 00}_5 \times 2$

f) times 2 dd 1234h, 5678h

⇒ memory 34 12 00 00 48 56 00 00 34 12 00 00 78 56 00 00

g) mov bh, 4Fh; 0111 1111 = 124

cmp bh, al; fictive subtraction that sets the flags $\begin{cases} CF=1, \text{ ah starts with 1-neg} \\ CF=0, \text{ ah starts with 0-pos} \end{cases}$

ror ah, 1; first bit goes on the last position

sar ah, 7; fill with the sign bit

ah = 1000 0000 ⇒ ~~1000 0000~~ ~~0001 00~~ 0000 0001, CF = sign bit

so this instructions

b) mov bh, 4Fh; bh = 0111 1111

cmp bh, al; fictive subtraction: $bh - al = 0111\ 1111 - al \Rightarrow$ ~~first bit~~ last bit of

ror ah, 1; ~~the~~ last bit of ah = sign bit (bit 0101 ...) al = sign bit ⇒ will set / $\begin{cases} CF=1, \text{ al starts with 1} \\ CF=0, \text{ al starts with 0} \end{cases}$

sar ah, 4; fill with the sign bit (bit bit bit...)

so these instructions convert al into a word for signed interpretation

movsx ax, al OR CBW

↓
loads in AX the signed-extended AL

$$\begin{array}{l}
 ax = 0000\ 0010 \\
 dx = 1111\ 1111
 \end{array}
 \begin{array}{l}
 1111\ 11 \\
 0101\ 0110 = 86 \\
 1111\ 1111 = -1 \\
 \hline
 10101\ 0101 \\
 \hline
 ah
 \end{array}$$

$$\begin{array}{r}
 256 \\
 1 \overline{) 16}^3 \\
 1536 \\
 \hline
 256 \\
 4096
 \end{array}$$

$$\begin{array}{r}
 16 \\
 6 \overline{) 96}^3 \\
 1280
 \end{array}
 \quad
 \begin{array}{r}
 256 \\
 5 \overline{) 1280}^3 \\
 72
 \end{array}$$

$$\begin{array}{r}
 4096 \\
 1280 \\
 \hline
 5376 \\
 96 \\
 \hline
 5472
 \end{array}$$

$$\begin{array}{r}
 16 \\
 5 \overline{) 80}^3
 \end{array}$$

$$\begin{array}{r}
 16 \\
 15 \overline{) 80}^3
 \end{array}$$

$$\begin{array}{r}
 16 \\
 24 \overline{) 0} \\
 86 \\
 \hline
 326
 \end{array}$$

$$\begin{array}{r}
 1 \\
 56 \\
 \hline
 FF \\
 55
 \end{array}$$

$$21:16 = 125$$

$$21 = F6$$

$$\begin{array}{l}
 0000\ 0000\ 1111\ 0110 \\
 0111\ 1011\ 0000\ 0000 \\
 \hline
 ah \quad al
 \end{array}$$

$$10h = 0001\ 0000$$

$$bh = \frac{0000\ 0011}{0001\ 0011}$$

a) mov eax, 200

mov ebx, 254h

idiv bl

$$200 = 08h = 1100\ 1000$$

$$\text{signed}(200) = 200$$

$$\text{unsigned}(200) = 200 \quad , 200 \text{ fits a dword}$$

$$254h = 4 + 5 \cdot 16 + 2 \cdot 256 = 4 + 80 + 512 = 596$$

$$\text{signed}(254h) = 596d$$

$$\text{unsigned}(254h) = 596d \quad , 596 \text{ fits a dword}$$

$$bl = 54h = 84d$$

$$\text{idiv bl} \Rightarrow 200 : 84 = 2 \text{ rest } 32 \quad \left\{ \begin{array}{l} AX = 2 = 0002h \\ DX = 32 = 0020h \end{array} \right.$$

CF and OF are not modified

b) mov ax, 256h

mov dx, -1

add ah, dh

$$256h = 598d$$

$$\Rightarrow \text{unsigned}(256h) = 598d$$

$$\left\{ \begin{array}{l} \text{signed}(256h) = 598d \\ \text{signed}(256h) = 598d \end{array} \right. , 598 \text{ fits a word}$$

$$\left(\begin{array}{l} 598 \in [0, 2^{16}-1] \\ 598 \in [-2^{16}, 2^{16}-1] \end{array} \right)$$

$$\left. \begin{array}{l} \Rightarrow ah = 56h \\ \quad = 86d \\ ah = 02h = 2d \end{array} \right\}$$

$$-1 = 2^{\text{'s}}(1)$$

$$2^{\text{'s}}(0000\ 0000\ 0000\ 0001) = 1111\ 1111\ 1111\ 1111 = \text{FFFF}h$$

$$\Rightarrow \text{unsigned}(-1) = 2^{16}-1 = 65535d \Rightarrow dh = \text{FF}h = 255$$

$$\text{signed}(\text{FFFF}h) = (-1)d$$

unsigned addition: add ah, dh; ah $\leftarrow 2+255 = 254 \notin [0, 255] \Rightarrow$

$$\Rightarrow CF = 1$$

$$\begin{array}{r}
 \begin{array}{cc}
 1111 & 11 \\
 0000 & 0010 + \\
 \hline
 1111 & 1111 \\
 1 | 0000 & 0010 \quad \Rightarrow CF = 1 \text{ (transp. dig. outside)} \\
 \hline
 \end{array} \\
 OF = 0 \quad (2 + (-1)) = 1 \in [-127, 127]
 \end{array}$$

$$\Rightarrow ah = 01h = 1d$$

$$c) \text{ mov ax, } \sim(16h/32)$$

$$\begin{array}{l}
 16h = 0001 \ 0110 \ b \\
 32 = 0010 \ 0000 \ b \\
 (2^5)
 \end{array}
 \left. \vphantom{\begin{array}{l} 16h \\ 32 \end{array}} \right\} \Rightarrow 16h/32 = 0011 \ 0110 \ b$$

$$\Rightarrow \sim(16h/32) = 1100 \ 1001 \ b$$

$$\Rightarrow ax = \text{dddd dddd } 1100 \ 1001 = \text{c9h} = 201d \text{ (signed and unsigned)}$$

$$\text{mov bx, } 2000h \gg 4$$

$$\begin{array}{l}
 FFc9h = 65481 \text{ (unsigned)} \\
 = -54 \text{ (signed)} \\
 2000h = \text{0010 } 0000 \ 0000 \ 0000 = \underbrace{0000 \ 0010}_{bh} \ 0000 \ 0000 = 0200h
 \end{array}$$

~~imul bh~~

$$\text{imul bh} ; DX: AX \leftarrow AX * BH \text{ (signed)}$$

$$AX * BH = 201 * 2 = 402d \text{ (fits a word)} \Rightarrow CF = OF = 0$$

$$402d =$$

~~imul bh~~

$$ah = c9h = 201d = -55d \text{ sig.}$$

$$bh = 02h = 2d$$

$$\text{signed: } -55 * 2 = -110d = 6535 - 110 = 65426 \text{ unsigned}$$

$$\text{unsigned: } 201 * 2 = 402$$

$$CF = OF = 0 \quad (b * b \rightarrow b)$$

d) mov ax, 21 << 4

mov bh, 10h ^ 3

sub bh, al

$$ax = 21 = 0000\ 0001\ 0101\ b = 0015h$$

$$0000\ 0001\ 0001\ 0101$$

$$21 \ll 4 = 0000\ 1010\ 1000\ 0000 = 0$$

$$21 \ll 7 = 0000\ 1010\ \underbrace{1000\ 0000}_{al} = 0A80h = 8 \cdot 16 + 10 \cdot 256 = 2560 + 128 = 2688\ d$$

(signed and unsigned)

$$10h = 0001\ 0000\ b$$

$$3 = 0000\ 0011\ b$$

$$10h \wedge 3 = 0001\ 0011 = 13h = 19\ d$$

(signed and unsigned)

$$\Rightarrow \underline{bh = 13h = 19d}$$

$$al = 1000\ 0000 = 2^7 = 128\ d \text{ (unsigned)}$$

$$-127\ d \text{ (signed)}$$

unsigned : sub bh, al ; bh = 19 - 128 (+256) = 144 d (unsigned)

signed : sub bh, al ; bh = 19 - (-127) = 146 d (signed) $\Rightarrow \overline{CF} = 1$

$$146 \notin [-128, 127] \Rightarrow \underline{\overline{OF} = 1}$$

e) stb bh, 8

add bx, ax

mov eax, ebx $\Leftrightarrow$ lea eax, [ebx]

EXAM - 2022

I Call, code, Entry code, Exit code
CDECL, STDCALL, ASM-ASM, ASM-C

II.

a1 dd 'abcdefh', abcdefh

'abcdefh' = '0' - as a dword $\Rightarrow 30h = 00\ 00\ 00\ 30 \Rightarrow \text{mem: } 30\ 00\ 00\ 00$
'a' - as a dword $\Rightarrow$

'b' = 97 = 61h $\Rightarrow 00\ 00\ 00\ 61 \Rightarrow \text{mem } 61\ 00\ 00\ 00$
'c' = 99 = 63h $\Rightarrow 00\ 00\ 00\ 63 \Rightarrow$

'd' = 100

'f' = 102 = 66h $\Rightarrow 00\ 00\ 00\ 66$

'h' = 104 = 68h $\Rightarrow 00\ 00\ 00\ 68$

~~00 00 00 30~~
 $\Rightarrow$ 00 00 00 30 'd' 00 00 00 61 'a' 00 00 00 62 'b' 00 00 00 63 'c' 00 00 00 66 'f' 00 00 00 68 'h'
00 00 00 65 'e' 00 00 00 66 'f' 00 00 00 68 'h' EF CD AB 00

abcdefh $\Rightarrow$ memory EF CD AB 00
00abcdef

a2 dw 'abcdefh', 3/6

'abcdefh' = 0030 0061 0062 0063 0064 0065 0066 0068

3 = 0000 0011
6 = 0000 0110 } $\Rightarrow 3/6 = 0000.0111 = 07h \Rightarrow \text{mem: } 04\ 00$

a3 dw \$-a2, a2-a1

\$-a2 = 18 (18 bytes between \$ and a2)

MEMORY LAYOUT

00401000	a1	00 00 00 30 00 00 00 61 00 00 00 62 00 00 00 63
00401010		00 00 00 64 00 00 00 65 00 00 00 66 00 00 00 68
00401020	E7 CB AB 00	00 61 00 62 00 63 00 64 00 65
00401030	00 66 00 68	04 00



a6 dd \$+a2-1, a2 ; syntax error addition of addresses not allowed

a7 dd 256h ^ 256, 256256h | 56 03 00 00 56 62 25 00

$$256h = 0000\ 0256h = 0000\ 0000\ 0000\ 0000\ 0000\ 0010\ 0101\ 0110 \Rightarrow$$

$$256 = 0000\ 0d10h = 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0000\ 0000$$

$$\Rightarrow 256h \wedge 256 = 5 * (0000) 00\ 11\ 0101\ 0110 = 0000\ 0356h \Rightarrow mem: 56\ 03\ 00\ 00$$

$$256256h = 00256256h \Rightarrow mem: 56\ 62\ 25\ 00$$

a8 dd (\$-a7)+(a9-\$), -256 | 10 00 00 00 00 FF FF FF

$$\$ - a7 = 2 \text{ (bytes)}$$

$$2 = 0000\ 0002h$$

$$a9 - \$ = 8 \text{ (bytes)}$$

$$(\$ - a7) + (a9 - \$) = 10 = 0000\ 0010h \Rightarrow mem: 10\ 00\ 00\ 00$$

$$-256 = 2^8(256)$$

$$\frac{256}{2^8} = 1 \Rightarrow 256 = 0000\ 0000\ 0000\ 0000\ 0000\ 0001\ 0000\ 0000 \Rightarrow$$

$$\Rightarrow -256 = FFFFFFFF00h \Rightarrow mem: 00\ FF\ FF\ FF$$

$$\Rightarrow mem: 00\ FF\ FF\ FF$$

II. a1 dd 'abcdefh', oabedefh | 30 61 62 63 64 65 66 68 EFCDAB 00

$$'a' = 97 = 61h$$

$$oabedefh = 00ABED EF$$

$$a2 \text{ dw 'oabedefh', 3|6 | 30 61 62 63 64 65 66 68 07 00}$$

$$\left. \begin{array}{l} 3 = 0000\ 0000\ 0000\ 0011 \\ 6 = 0000\ 0000\ 0000\ 0110 \end{array} \right\} \Rightarrow 3|6 = 0000\ 0000\ 0000\ 0111 = 0007h$$

$$\Rightarrow \text{mem: 07 00}$$

$$a3 \text{ dw } \phi - a2, a2 - a1 \mid 0A\ 00\ 0E\ 00$$

$$\phi - a2 = 10 \text{ bytes} = 000Ah \Rightarrow \text{mem: } 0A\ 00$$

$$a2 - a1 = 12 \text{ bytes} = 000Ch \Rightarrow \text{mem: } 0E\ 00$$

$$a4 \text{ dw } 129 \gg 1, -129 \ll 1 \mid \text{40 FE}$$

$$129 \gg 1, 129 = 1000\ 0001 \Rightarrow 129 \gg 1 = 0100\ 0000 = 40h$$

$$-129 = 2^5(129) \text{ (unsign. ext.)} = 2^5(0000\ 0000\ 1000\ 0001) = 1111\ 1111\ 0111\ 1111 = FFFFh$$

$$= 4Fh$$

$$0111\ 1111 \ll 1 = 1111\ 1110 = FEh$$

$$a5 \text{ dw } a2 - a4, \sim(a2 - a4) \mid F2\ FF\ 0D\ 00$$

$$a2 - a4 = -14 \text{ (14 bytes between)}$$

$$-14 = 2^5(14)$$

$$14 = 0000\ 0000\ 0000\ 1110 = 000Eh \Rightarrow \text{mem: } 0E\ 00$$

$$\sim(a2 - a4) = \sim(-14)$$

$$\Rightarrow -14 = 1111\ 1111\ 1111\ 0010 = FFF2h \Rightarrow \text{mem: } F2\ FF$$

$$\sim(a2 - a4) = \sim(1111\ 1111\ 1111\ 0010) = 0000\ 0000\ 0000\ 1101 = 000Dh$$

$$\Rightarrow \text{mem: } 0D\ 00$$

2b) push ebp
 mov ebp, esp
 pop eax; mov eax, ebp
 shl eax, 1; $eax = eax * 2$
 sub eax, 5; $\leftarrow$ mov ebx, esp
~~add~~ ebx, eax; ~~$ebx = eax + 5 = ebp + 1 + 5$~~ $ebx = esp + eax = esp + 2$
 mov esp, ebp

III. a) mov ax, 0100h
 mov bx, 1000 + 10b
 idiv bl

~~1000~~ = 0100h = 256d (signed
 and unsigned = 6)

$1000 + 10b = 1000 + 2 = 1002 = \cancel{0000\ 0001\ 1100\ 0110} = 6 + 64 + 128 = 198d$
 (unsigned)

$-2^5(198) = -58$

03EAh $\Rightarrow$ bl = EA = 234d (unsigned)
 $= -22d$ (signed)

$2^5(198) = 0011\ 1010 = 10 + 16 + 32 = 58$

signed: idiv bl $\Rightarrow$ quotient in al, remainder in ah

$256 \cdot (-58) = -4$ rest 24, $al = -4 = 1111\ 1100 = FCh$

$4 = 0000\ 0100$

$ah = 24 = 18h$

unsigned idiv bl

$256 : 198 = 1$ r 58 $\Rightarrow al = 1 = 01h$

$ah = 58 = 3Ah$

signed idiv bl: $256 \cdot (-22) = -11$ r E(14)

$al = -11$

$ah = 14 = Eh$

unsigned idiv bl: $256 : 234 = 1$ r 22
 $al = 1$ $ah = 22$

$$\begin{array}{r} 234 : 16 = 14 \\ 16 \\ \underline{144} \\ 90 \\ \underline{64} \\ 26 \end{array}$$

$$\begin{array}{r} 1002 : 256 = 3 \\ 256 \\ \underline{768} \\ 234 \end{array}$$

$$\begin{array}{r} 16 \\ 14 \\ \underline{14} \\ 2 \end{array}$$

$$\begin{array}{r} 256 : 58 = 4 \\ 58 \\ \underline{232} \\ 24 \end{array}$$

$$\begin{array}{r} 256 : 198 = 1 \\ 198 \\ \underline{198} \\ 58 \end{array}$$

$$\begin{array}{r} 256 \\ 234 \\ \underline{22} \end{array}$$

$$\begin{array}{r} 256 : 22 = 11 \\ 22 \\ \underline{242} \\ 14 \end{array}$$

MEMORY LAYOUT

~~00401000~~

00401000	^{a1} 30 61 62 63 64 65 66 68 EF CD AB 00 ^{a2} 30 61 62 63
00401010	64 65 66 68 04 00 ^{a3} 0A 00 0E 00 ^{a4} 40 FE ^{a5} 02 FF 0D 00
00401020	^{a*} 56 03 00 00 56 62 25 00 ^{a3} 0E 00 00 00 00 FF FF FF
00401030	^{a3} 01 FF 80 FF 28 10 80 FF 28 04 80 FF 28 04 80 FF
	28 04 80 FF ^{all} 04 16 10

a3 dw -255, -128 | 01 FF 80 FF

$$-255 = 2^8(255)$$

$$255 = \cancel{0000} \cancel{0000} 0000$$

$$255 = 0000 0000 1111 1111 \Rightarrow -255 = 1111 1111 0000 0001 = FF01h$$

$\Rightarrow$ mem: 01 FF

$$-128 = 2^7(128)$$

$$128 = 0000 0000 1000 0000 \Rightarrow -128 = 1111 1111 1000 0000 = FF80h$$

$\Rightarrow$ mem: 80 FF

a10 times 4 dw 128h, -128 | 28 ~~04~~ 80 FF 28 ~~04~~ 80 FF 28 ~~04~~ 80 FF

$$128h = 0128h \Rightarrow \text{mem: } 28 \text{ } \cancel{04}$$

$$-128 = 2^7(128) = FF80h \Rightarrow \text{mem: } 80 \text{ FF}$$

a11 db a3 ~~fix~~ syntax error, offsets not rep on 2bits.

$$a3 = \cancel{00401014} \Rightarrow \text{last byte} = 14h$$

a12 dw a3 | 16 10

$$a3 = 0040101\cancel{6} \Rightarrow \text{last word } 1017h \Rightarrow \text{mem: } 1\cancel{6} 10$$

b) mov ah, 0cdh

mov al, 0ebh

add ah, al

$$ah = \text{cd} = 192 \text{ d (unsigned)} = 1100 \ 1101$$

$$= -64 \text{ d (signed)}$$

$$256 - 192 = 64$$

$$al = \text{eb} = 235 \text{ d (unsigned)} = 1110 \ 1011$$

$$= -27 \text{ d (signed)}$$

add ah, al

$$\begin{array}{r} 1100 \ 1101 \\ + 1110 \ 1011 \\ \hline 1011 \ 1000 \end{array}$$

$$\Rightarrow ah = 1011 \ 1000 = B8h = 176 + 8 = 184 \text{ (unsigned)}$$

CF = 1 (a transp. digit outside...)

$$= -42 \text{ (signed)}$$

OF = 0 (no overflow on signed)

c) mov ax, 1010h

mov bx, 1111b

mul bl

$$1010h \Rightarrow al = 10h = 16$$

$$1111b = 15d$$

$$\text{mul bl} \Rightarrow al * bl = 16 * 15 = 240 \text{ (result in ax)} = F0h \ 00F0h$$

$$\left\{ \begin{array}{l} al = F0h = -16 \text{ (signed)} \\ ah = 00h \end{array} \right.$$

CF = 0, OF = 0 (b * b = b) no overflow on multiplication

d) mov dh, 200

mov ch, 62h

sub dh, ch

$$200 = C8h = 1100 \ 1000b$$

$$62h = 96 + 2 = 98d = 0110 \ 0010b$$

$$\text{sub dh, ch} \Rightarrow \begin{array}{r} 1100 \ 1000 \\ - 0110 \ 0010 \\ \hline 0010 \ 0010 \end{array}$$

$$= 34 \text{ (sig + uns)}$$

$$\left\{ \begin{array}{l} CF = 0 \text{ (no transp. dig.)} \\ OF = 1 \text{ (neg - pos = pos)} \end{array} \right.$$

push ebp

mov ebp, esp; mov esp, ebp

pop eax; mov eax, ebp

shl eax, 1; $eax \leftarrow 2 * eax$

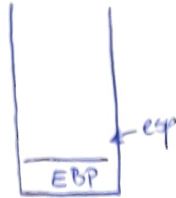
sub eax, 5; $eax \leftarrow 2 * eax - 5$

mov ebx, esp; ~~ebx $\leftarrow$ esp~~ $\xrightarrow{\text{point to ebp}}$ ebx $\leftarrow$ ebp

add ebx, eax; $ebx \leftarrow ebp + eax = ebp + 2 * eax - 5$

mov esp, ebp

($\Rightarrow$) ^{lec-}~~mov~~ ebx, [esp + 2 * eax - 5]





00401000
00401010
00401020

II.

$$-103 = 2^1 s(103)$$

$$00401000 + 36$$

$0040\ 1010 \xrightarrow{16} 0040\ 1020 \xrightarrow{16} 0040\ 1030 \xrightarrow{4} 0040\ 1034$

~~$$203 = 64h = 0000\ 0000\ 0000\ 0000\ 0110\ 0111 \Rightarrow -103 = 1111 * 4\ 1001\ 1001 = FFFFFFFF99h$$~~

103 = ~~00~~ 00 00 64 h = FF FF FF 99 h $\Rightarrow$ mem: 99 FF FF FF

$\text{offset}(ac) = 00401034 \Rightarrow \text{memory: } 34104000$

Q2. du $1 \nmid 2, 3 \mid 4, 5 \nmid 6, 1 \mid 2, 3 \nmid 4, 5 \nmid 6$

$$\left. \begin{array}{l} 1 = 0000 \ 0000 \ 0000 \ 0001 \\ 2 = 0000 \ 0000 \ 0000 \ 0010 \end{array} \right\} \Rightarrow 1 \oplus 2 = 00 \dots 0 = 0000$$

$$\begin{array}{l} 3 = 0000 \ 0000 \ 0000 \ 0011 \\ 4 = 0000 \ 0000 \ 0000 \ 0100 \end{array} \Bigg\} \Rightarrow 3 \mid 4 = 0000 \ 0000 \ 0000 \ 0111 = 0004_k \Rightarrow$$

$\Rightarrow$ wenn: 07 00

$5 = 0000\ 0000\ 0000\ 0101$
 $6 = 0000\ 0000\ 0000\ 0110$

$\Rightarrow 5^6 = 000\dots0011 = 0003_{16} \Rightarrow \text{mem: } 03\ 00$

$$\left. \begin{array}{l} 1 = 0000\ 0000\ 0000\ 0001 \\ 2 = 0000\ 0000\ 0000\ \underline{0010} \\ \quad \quad \quad \underline{0011} \end{array} \right\} \Rightarrow 1|2 = 0003h \Rightarrow \text{mem: } 03\ 00$$

$$\left. \begin{array}{l} 3 = 0000 \ 0000 \ 0000 \ 0011 \\ 4 = 0000 \ 0000 \ 0000 \ 0100 \end{array} \right\} \Rightarrow 3^4 = 0003h \rightarrow \text{mem: } 03 \ 00$$

5 =

$6 =$

II a) segment data are 32 data = data

little endian $\Rightarrow$ 00 FF 56 02

x dw -256, 256h; FF00 0256 offset(x) = 0040 1000

y dw 256 / -256, 256h & 256; FF00 0000, offset(y) = 0040 1008
00 FF 00 00

z db \$-z, y-x; 00 02, offset(z) = 0040 1010 (?)

db 'y'-x, 'y-x'; 01 7F (code of '-') 70, offset = 0040 1012
 $\hookrightarrow$ ~~4 bits~~ diff distance $\text{addr}(x) - \text{addr}(y) = 4 \text{ bits} = 2 \text{ bytes}$

a db 512 >> 2, -512 << 2; 08 80, offset(a) = 0040 1015

b dw z-a, !(z-a) (syntax error, because ! operator can)

c dd (\$-b) + (d-\$), \$-2*y+3

d db -128, 128 ^ (-128); 00 00

e times 2 resw 6; 06 06

times 2 dd 1234h, 5678h; 3412 7856

(b) mov bh, 4Fh

cmp bh, AL

rcr AH, 1

sar AH, 4

(vreau să pun val din conținut AH)

$$\begin{array}{r} 256 \\ 60 \\ -196 \end{array}$$

$$\begin{array}{r} 200 \\ 256 \\ -56 \end{array}$$

$$200 = C8$$

$$32$$

$$\begin{array}{r} 400 \\ 256 \\ -144 \end{array}$$

$$\begin{array}{r} 16 \\ 256 \\ -16 \\ 240 \\ 16 \\ -16 \\ 224 \\ 16 \\ -16 \\ 208 \\ 16 \\ -16 \\ 192 \\ 16 \\ -16 \\ 176 \\ 16 \\ -16 \\ 160 \\ 16 \\ -16 \\ 144 \\ 16 \\ -16 \\ 128 \\ 16 \\ -16 \\ 112 \\ 16 \\ -16 \\ 96 \end{array}$$

$$\begin{array}{r} 0011 \ 0010 \\ 0011 \ 0010 \\ \hline \end{array}$$

$$\begin{array}{r} 20016 = 12 \\ 16 \\ 40 \\ 32 \end{array}$$

$$\begin{array}{r} 16 \\ 5 \\ 80 \end{array}$$

$$\begin{array}{r} 1100 \ 1000 \\ 1100 \ 1000 \\ \hline 1001 \ 0000 \\ 1010 \ 0000 \\ 1010 \ 0000 \\ \hline 0100 \ 0000 \end{array}$$

$$\begin{array}{r} 2^5 = 32 \\ 2^7 = 128 \end{array} \left. \begin{array}{l} 256 \\ 160 \end{array} \right\} \begin{array}{r} 96 \end{array}$$

III. a) mov eax, 200

mov ebx, 254h

idiv bl; the result will be stored in AX ~~100 = 64h~~ = 50d = 32h (quotient)

and remainder in DX = 0

CF = OF = 0 (the res. fit on a word)

$$\begin{array}{r} 1111 \ 11 \\ 0101 \ 0110 \\ 1111 \ 1111 \\ \hline 10101 \ 0100 \\ 2^6 \ 2^4 \ 2^2 \\ 64 \ 16 \ 4 \end{array}$$

b) mov ax, 256h; ax = ~~0101 0110~~ 0000 0010 0101 0110
mov dx, -1; dx = 1111 1111 1111 1111

add ah, dh; ah = ~~0101 0110~~ 0101 0100 CF = 1, OF = 0

b) mov ax, 256h; ax = 0000 0010 0101 0110b =

mov dx, -1; dx = 1111 1111 1111 1111b = FFFFh

add ah, dh; ah = 0101 0101b = 99h =

unsigned: 256h = 544d ~~256h~~ ah = 56h = 86d

unsigned: FFFFh = -1 $\Rightarrow$ dh = FFh = 255d ~~255d~~ 255

~~55h + FFh = 155h~~

56h + FFh = 86d + 255d = 341 = 155h doesn't fit byte AH $\Rightarrow$

$\Rightarrow$ AH = 55h and CF = 1 (because there's a transport digit outside the byte rep)

signed: $256h, ah = 56h = 86d$

signed: $FFFFh, dh = Fh = -1d$

$$56h + Fh = 86d + (-1)d = 85d =$$

$$56h + Fh = 155h = 34d$$

$$AH = 55 (0101\ 0101) = 85d \in [-128, 127] \Rightarrow OF = 0 \text{ (no overflow on signed interpretation)}$$

a) $\text{mov ax}, 21 \ll 4$

$\text{mov bh}, 10h \wedge 3$

$\text{sub bh}, al$

$$21 \ll 4; F6 \ll 4; 0000\ 0000\ 1111\ 0110 \ll 4 \Rightarrow 0111\ 1011\ 0000\ 0000$$

unsigned
 $\Rightarrow al = 0000\ 0000 = 00h$

$$10h \wedge 3 \Rightarrow 0001\ 0000 \wedge \begin{array}{r} 0000\ 0011 \\ 0001\ 0011 \end{array} \Rightarrow bh = 0001\ 0011 = 13h$$

$\text{sub bh}, al \Rightarrow \begin{array}{r} 0001\ 0011 \\ 0000\ 0000 \\ \hline 0001\ 0011 \end{array} = 13h \Rightarrow CF = 0 \text{ (no transp digit, res fit)}$
 $OF = 0 \text{ (res fit)}$

signed - the same

c) $\text{mov ax}, \sim(16h \mid 32)$

$\text{mov bx}, 2000h \gg 4$

imul bh

$$\begin{array}{r} 6+ \\ 16 \\ 32 \\ \hline 54 \end{array}$$

$$\begin{array}{l} 16h = 0000\ 0000\ 0001\ 0110 \\ 32 = 0000\ 0000\ 0010\ 0000 \end{array} \left. \vphantom{\begin{array}{l} 16h \\ 32 \end{array}} \right\} \stackrel{①}{\Rightarrow} 0000\ 0000\ 0011\ 0110 \Rightarrow \sim$$

① $\oplus 36h$

$$1111\ 1111\ 1100\ 1001b = FFC9h \Rightarrow ax = FFC9h = (\text{signed})$$

$$2000h \gg 4 \Rightarrow 0200h \Rightarrow bx = 0200h \Rightarrow bh = 02h = 2d$$

$$2's(1111\ 1111\ 1100\ 1001) = 0000\ 0000\ 0011\ 0111 = 55d \Rightarrow FFC9h = -55d \Rightarrow$$

$$dx \cdot ax = ax \cdot bh = -55 \cdot 2 = -110d \Rightarrow ax = -55$$

$$CF = OF = 0$$

e) `shl bx, 8`

`add bx, ax`

`mov eax, ebx` / `movzx eax, ebx` (loads in `eax` the unsigned version of `bx`)

bits 32

global start

extern exit, printf, scanf

import exit msvert.dll

import printf msvert.dll

import scanf msvert.dll

segment data use32 class = data

string times 100 db 0

number dd 0

formats db "%s", 0

formatnr db "%d", 0

segment code use 32 class = code

start:

```
push dword number  
push dword [formatnr] } read read nr  
call [scanf]  
add esp, 4*2
```

```
push dword [number]  
push dword [formatnr] } print nr  
call [printf]  
add esp, 4*2
```

loop-read:

```
push dword number  
push dword [formatnr]  
call [scanf]  
add esp, 4*2
```

cmp dword [number], 0

je end-prog

jmp loop-read

$$200 = \text{C8h}$$

$$\text{eax} = 0000\ 0000\ 1100\ 1000$$

$$\text{signed}(\text{eax}) = 0000\ 0000\ 1100\ 1000$$

$$\text{ebx} = 0000\ 0010\ 0101\ 0100$$

$$\text{signed}(\text{bx}) = 0100 = 4$$

$$0000\ 0001$$

$$1111\ 1111$$